



(A) 3 (B) 15 (C) 9 (D) -3

(e) [2 points] What is the maximum number of swaps that can be performed in the Selection Sort algorithm?

(A) n (B) $n - 1$ (C) $n - 2$ (D) 1

(f) [2 points] Let A be a square matrix of size $n \times n$. Consider the following algorithm. What is the expected output?

```
C = 500
for i = 1 to n do
    for j = 1 to n do {
        Temp = A[i][j] + C
        A[i][j] = A[j][i]
        A[j][i] = Temp - C
    }
for i = 1 to n do {
    for j = 1 to n do
        Output(A[i][j]);
    Output("\n");
}
```

(A) Transpose of matrix A

(B) The matrix A itself

(C) Adding 500 to the diagonal elements

(D) Adding 500 to the diagonal elements and subtracting 500 from anti-diagonal elements of A

(g) [2 points] Suppose a circular queue of capacity $(n - 1)$ elements is implemented with an array of n elements. Assume that the insertion and deletion operation are carried out using REAR and FRONT as array index variables, respectively. Initially, $\text{REAR} = \text{FRONT} = 0$. The conditions to detect queue **Full** and queue **Empty** are

(A) **Full:** $(\text{FRONT} + 1) \bmod n == \text{REAR}$, **Empty:** $\text{REAR} == \text{FRONT}$

(B) **Full:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$, **Empty:** $(\text{FRONT} + 1) \bmod n == \text{REAR}$

(C) **Full:** $\text{REAR} == \text{FRONT}$, **Empty:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$

(D) **Full:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$, **Empty:** $\text{REAR} == \text{FRONT}$

(h) [2 points] A three dimensional array in C language is declared as $\text{int } A[x][y][z]$. Here, the address of an item at the location $A[p][q][r]$ can be computed as follows (where w is the word length of an integer):

(A) $\&A[0][0][0] + w(y * z * p + x * q + r)$

(B) $\&A[0][0][0] + w(x * y * p + y * q + r)$

(C) $\&A[0][0][0] + w(y * z * p + z * q + r)$

(D) $\&A[0][0][0] + w(x * y * z * p + y * z * q + r * z)$

$$n^0 < n! < n^{2n}$$

4. [5 points] Given a collection of algorithms that runs on $O(n^{10})$, $O(n^{2n})$, $O(10000)$, $O(\log n)$, $O(n!)$, $O(n \log n)$, $O(n)$, $O(n \log \log n)$ order the algorithms **from fastest to slowest** when n very large.
5. [6 points] Given a circular queue of size 4 (i.e., $\text{MAX_QUEUE_SIZE} = 4$), what is the **front** and **rear** values become when we do the following commands? (Initial circular queue setting: $\text{front} = \text{rear} = 0$)
add A, add B, add C, add D, delete, add E, add F, add G, delete, delete, add H, add I, add J
6. [10 points] Use **Big-Theta** notation,

- (a) [3 points] Suppose the recursive function $T(n)$ calls $T(n-1)$ **five times** within its body. Also assume the recursion would **end when $n=0$** . Estimate the **time complexity of $T(n)$** .

For the following question (b)-(c),

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++)
    for (j = 2; j <= n; j = j * 2)
        k = k + n / 2;
```

Code 1

```
int a = 0, i = N;
while (i > 0) {
    a += i;
    i /= 2;
}
```

Code 2

- (b) [3 points] Estimate the time complexity of **Code 1**.
- (c) [4 points] Estimate the time complexity of **Code 2**.
7. [10 points] Consider the following recursive functions.

<pre>int func1(int a, int b){ if (a % b == 0) return b; else return func1(b, a % b); }</pre> $52 \ 38$ $38 \ 14$ $14 \ 10$ $10 \ 4$ $4 \ 2$	<pre>int func2(int a, int b){ if (a == 0) return b + 1; else if (b == 0) return func2(a - 1, 1); else return func2(a - 1, func2(a, b - 1)); }</pre> $1 \ 2 \ 3 \ 4 \ 5$ $1 \ 2 \ 3 \ 4$ $1 \ 2 \ 3$ $1 \ 2$ 1
--	--

- (a) [5 points] What is the output given by $\text{func1}(90, 52)$?
- (b) [5 points] What is the output given by $\text{func2}(2, 3)$?
8. [10 points] What are the values of **a** to **j** in the following programs:

Statement	s/e	Frequency
<code>void func1(int a[][n], int b[][n], int d[][n])</code>		
<code>{</code>		
<code>int i, j, k;</code>		

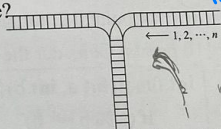
for (i = 0; i < n; i++)	1	a
for (j = 0; j < n; j++) {	1	b
c[i][j] = 10;	1	c
for (k = 0; k < n; k++)	1	d
d[i][j] += a[i][k] * b[k][j] - 20;	1	e
}		
}		
Total steps = $\Theta(f)$		

Statement	s/e	Frequency
float func2(float list[], int n)		
{		
If (n)	1	g
return func2(list, n-1) * list[n-1];	1	h
return list[0];	1	i
}		
Total steps = $\Theta(j)$		

9. [11 points] Consider the railroad switching network. Railroad cars numbered 1, 2, ..., n are at the right. Each car is brought into the stack and removed at any time.

卡特蘭數: $\frac{(2n)!}{(n+1)!n!}$

- (a) [2 points] For $n = 9$, is the new order 3 4 5 2 1 8 9 6 7 possible?
 (b) [2 points] How many possible permutations for $n = 4$.
 (c) [3 points] How many possible permutations for $n = 5$?
 (d) [4 points] How many possible permutations for $n = 6$?



10. [12 points] In C/C++ language, the standard way of writing c expressions is known as **infix** notation because in it we place a binary operator in-between its two operands. Instead compilers typically use a parenthesis-free notation referred to as **postfix**. Another expression form that is easy to evaluate and is parenthesis-free is known as **prefix**. In Prefix notation, the operators precede their operands.

- (a) [4 points] Write the **postfix** form of the infix expressions:

$$A + (((B / C) * (D - A)) / F) / H - (E * G)$$

- (b) [5 points] Write the **prefix** form of the postfix expressions:

$$C \mid B \mid A > D \mid E < \mid \mid \&\& \mid C \mid E < \mid \mid >$$

$((C \&\& (B > A)) (D < E)) \mid ((C < E))$

- (c) [3 points] What is the value of the following postfix expression?

$$6 \ 3 \ 4 \ * \ + \ 2 \ / \ 2 \ 8 \ 2 \ + \ * \ 8 \ * \ -$$

$$\frac{4}{3} = 6$$

$$\frac{12}{6} = 2$$

$$\frac{18}{9} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

$$\frac{20}{10} = 2$$

3.

(e) B

(f) B → 仔細 Trace Code!

(g) D → Chapter. 3 P.18-19

$$(h) [p \times y \times z + q \times z + r] \times w + \&A[0][0][0] \\ = \&A[0][0][0] + w(p \times z + q \times z + r) = (c)$$

4.

$$O(100000) < O(\log n) < O(n) < O(n \log \log n) < O(n \log n) \\ < \underline{O(n^{10}) < O(n!) < O(n^{2n})}$$

開 n 次方根比較! $(n^{10})^{\frac{1}{n}} = 1$, $(n!)^{\frac{1}{n}} = \frac{n}{e}$, $(n^{2n})^{\frac{1}{n}} = n^2$

5.

cmd	front	rear	q[0]	q[1]	q[2]	q[3]
-	0	0				
add A	0	1		A		
add B	0	2		A	B	
add C	0	3		A	B	C
add D	0	3		A	B	C
delete	1	3			B	C
add E	1	0	E		B	C
add F	1	0	E		B	C
add G	1	0	E		B	C
delete	2	0	E			C
delete	3	0	E			
add H	3	1	E	H		
add I	3	2	E	H	I	
add J	3	2	E	H	I	

front = 3

rear = 2

6、

$$T(n) = aT(n-1) + O(1)$$

(avg $O(1)$ operate)- 次 n 中会呼叫 a 个 $T(n-1)$ + 加·減·乘·除·return.....

$$\Rightarrow T(n) = \Theta(a^n)! \rightarrow \text{背起來!}$$

常見變體:

$$- T(n) = aT(n-1) + O(n^d)$$

* d is a constant (O include)

$$\Rightarrow \Theta(a^n)$$

$$- T(n) = aT(n-c) + O(1)$$

$$\Rightarrow \Theta(a^{\frac{n}{c}})$$

$$- T(n) = aT(\frac{n}{b}) + O(1)$$

$$\bullet a > 1 \Rightarrow \Theta(n^{\log_b a})$$

$$\bullet a = 1 \Rightarrow \Theta(\log_b n)$$

(a)

```
int T(int n){
    if(n==0) return 1;
    else return T(n-1)+
               T(n-1)+
               T(n-1)+
               T(n-1);
}
```

}

$$T(n) = 5T(n-1) + O(1); \Rightarrow \Theta(5^n)_{\#}$$

s/e

(b)

```
int i, j, k = 0;
```

```
for(i = n/2; i <= n; i++) {
```

```
    for(j = 2; j <= n; j = j*2) {
```

```
        k = k + n/2;
    }
```

}

}

$$\frac{n}{2} + 1$$

$$n - 1$$

$$1$$

$$\frac{n}{2} \times n \Rightarrow \Theta(n^2)_{\#}$$

(c)

```
int a = 0, i = N;
```

```
while(i > 0) {
```

```
    a += i;
```

```
    i /= 2;
```

}

 $\Rightarrow$ 將會一直跑, 直到 $2^t = N$ 時

$$\Rightarrow \log(N)$$

/ = 2 ; * = 2 都是 log! Ans: $\Theta(\log N)$

8

(a) $\text{func1}(90, 52)$, 問 output?

$$f(90, 52) \rightarrow f(52, 38) \rightarrow f(38, 14) \rightarrow f(14, 10) \rightarrow f(10, 4)$$

$$\rightarrow f(4, 2) \rightarrow \text{Output} = 2 \times$$

(b) $\text{func2}(2, 3)$, 問 Output?

<key>

$$f(a, b) = b + 1$$

a \ b	0	1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	
2	3	5	7	9					

$$\text{Ans: } f(2, 3) = 9 \times$$

7.

```
void func1(int a[][n], int b[][n], int d[][n]){
{
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            c[i][j] = 10;
            for (k = 0; k < n; k++)
                d[i][j] += a[i][k] * b[k][j] - 20;
        }
    }
```

for (i = 0; i < n; i++)	1	a	$n+1$
for (j = 0; j < n; j++) {	1	b	$n \times (n+1) = n^2 + n$
c[i][j] = 10;	1	c	n^2
for (k = 0; k < n; k++)	1	d	$n^2(n+1) = n^3 + n^2$
d[i][j] += a[i][k] * b[k][j] - 20;	1	e	n^3
}			
}			
Total steps = $\Theta(f)$			

Statement	s/e	Frequency
float func2(float list[], int n)		
{		
If (n)	1	g $n+1$
return func2(list, n-1) * list[n-1];	1	h n
return list[0];	1	i 1
}		
Total steps = $\Theta(j)$		

func1 Total Steps: $(n+1) + (n^2+n) + n^2 + (n^3+n^2) + n^3 = 2n^3 + 3n^2 + 2n + 1$
 $\Rightarrow \Theta(n^3), f = n^3$

func2 Total Steps: $(n+1) + n + 1 = 2n + 2$
 $\Rightarrow \Theta(n), j = n$

9.

(a) impossible

$$(b) \frac{8!}{5!4!} = \frac{8 \times 7 \times 6 \times 2}{4 \times 3 \times 2 \times 1} = 14 \neq$$

$$(c) \frac{10!}{6!5!} = \frac{10 \times 9 \times 8 \times 7}{5 \times 4 \times 3 \times 2 \times 1} = 42 \neq$$

$$(d) \frac{12!}{7!6!} = \frac{12 \times 11 \times 10 \times 9 \times 8}{6 \times 5 \times 4 \times 3 \times 2 \times 1} = 132 \neq$$

調度員問題，幾種可能性

 $\rightarrow$ 卡塔蘭數 $\frac{(2n)!}{(n+1)!n!}$

10.

$$(a) \left\{ A + \left[\left(\begin{pmatrix} B & C \end{pmatrix} * (D-A) \right) / F / H \right] - (E * G) \right\} + EG^* -$$

$$ABC / DA^* F / H / + EG^* -$$

(b) 不考

(c) ~~6~~ ~~3~~ ~~4~~ ~~*~~ ~~7~~ ~~2~~ ~~X~~ ~~2~~ ~~8~~ ~~2~~ ~~*~~ ~~*~~ ~~8~~ ~~*~~ -

4			2		
3	12	2	8	10	8
6	18	9	20		160

$$9 - 160 = -151 \neq$$

上课洩題:

- big O 算法 (會給定義)

$n^2 + 3n + 3 = O(n^2)$, show the statement is correct.

- 陣列記憶體位址算法 CH.2 P.46
(1D, 2D, 3D)

- infix, postfix

- consider program's size & frequency, find complexity

- stack, Queue, circular queue 表格

- A mazing Problem

- Binary Search

- fast transpose matrix CH.2 P.32

CHEATING IS PROHIBITED!

- let $f(n) = n^2 + 3n + 3$, $g(n) = n^2$

take $c=4$, for all $n \geq 2$, $f(n) \leq cg(n)$

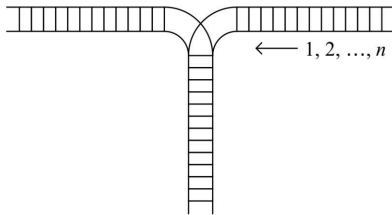
$\therefore n^2 + 3n + 3 \neq$

Circular Queue

Full: $\text{front} == (\text{rear} + 1) \% \text{MAX}$

Empty: $\text{front} == \text{rear}$

1. (10%) Consider the railroad switching network. Railroad cars numbered $0, 1, 2, \dots, n-1$ are at the right. Each car is brought into the stack and removed at any time. For instance, if $n = 3$, we could move in 0, move in 1, move in 2, and then take the cars out, producing the new order 2, 1, 0. Find all possible permutations for $n = 4$.



$$\frac{8!}{5!4!} = \frac{8 \times 7 \times 6^2}{4 \times 3 \times 2 \times 1} = 14$$

totals 14 way

0 1 2 3 ✓
 0 1 3 2 ✓
 0 2 1 3 ✓
 0 2 3 1 ✓
 0 3 2 1 ✓

2 1 0 3 ✓
 2 1 3 0 ✓
 2 3 1 0 ✓
 3 2 1 0 ✓

1 0 2 3 ✓
 1 0 3 2 ✓
 1 2 0 3 ✓
 1 2 3 0 ✓
 1 3 2 0 ✓